

Cấu trúc:

/api/v1

Public API

Auth

Cơ chế access token + refresh token

(access token lưu localStorage / refresh token lưu cookie - chỉ đính kèm với api refresh)

POST /api/v1/auth/login — **rate limit: 5 req / 15 phút / IP** (429 nếu vượt)

POST /api/v1/auth/signup — **rate limit: 3 req / 1 giờ / IP** (429 nếu vượt)

POST /api/v1/auth/refresh (kèm cookie refresh token)

POST /api/v1/auth/logout

Cơ chế OTP: cache Redis, TTL 10 phút. Mỗi mục đích có endpoint riêng:

POST /api/v1/auth/verify-email/send — gửi OTP kích hoạt tài khoản; body { email }

POST /api/v1/auth/verify-email — xác thực OTP kích hoạt; body { email, otp }

POST /api/v1/auth/forgot-password — gửi OTP đặt lại mật khẩu; body { email }

POST /api/v1/auth/reset-password — đặt lại mật khẩu với OTP; body { email, otp, newPassword }

Lessons công khai

- GET /api/v1/lessons — danh sách lesson đã publish; hỗ trợ filter theo tagId, ceFrLevelId, mode, q, page, limit.

Decks công khai

- GET /api/v1/decks — danh sách deck hệ thống (ownerType = system) đã publish; hỗ trợ filter theo tagId, ceFrLevelId, q, page, limit. Không bắt buộc đăng nhập. Deck cá nhân của user truy cập qua /users/me/decks.

Cần token API

Lessons

- GET /api/v1/lessons/{lessonId} — chi tiết một lesson đã publish. Yêu cầu đăng nhập.
- GET /api/v1/lessons/{lessonId}/segments — danh sách segment của lesson, sắp xếp theo startMs tăng dần. Yêu cầu đăng nhập.
- GET /api/v1/lessons/{lessonId}/segments/{segmentId} — chi tiết một segment. Yêu cầu đăng nhập.

Profile

- GET /api/v1/users/me — lấy thông tin user hiện tại.

- PUT /api/v1/users/me — cập nhật hồ sơ cá nhân như name, avatarUrl.

Metadata công khai

- GET /api/v1/cefr-levels — lấy danh sách level CEFR để filter lesson/deck.
- GET /api/v1/tags — lấy danh sách tag. Tùy chọn ?usedBy=lesson|deck để chỉ lấy tag đang được lesson/deck (đã publish) sử dụng; không truyền = tất cả tag.

Decks

- GET /api/v1/decks/{deckId} — chi tiết deck.
- GET /api/v1/decks/{deckId}/topics — danh sách topic trong deck kèm tiến độ topic của user.
- GET /api/v1/decks/{deckId}/topics/{topicId}/cards — danh sách card trong topic kèm trạng thái học của user. Card hệ thống trả về thêm `quizOptions [{word, isCorrect}]` để render trắc nghiệm.

Deck của user (ownerType = user)

- GET /api/v1/users/me/decks — danh sách deck do user hiện tại sở hữu; hỗ trợ filter q, page, limit.
- POST /api/v1/users/me/decks — tạo deck mới thuộc sở hữu user hiện tại (luôn published; body chỉ gồm title, description). Tối đa 3 deck/user.
- GET /api/v1/users/me/decks/{deckId} — chi tiết một deck của user.
- PUT /api/v1/users/me/decks/{deckId} — cập nhật deck (chỉ chủ sở hữu).
- DELETE /api/v1/users/me/decks/{deckId} — xóa deck kèm toàn bộ topic và card bên trong (chỉ chủ sở hữu).

Topics trong deck của user

- GET /api/v1/users/me/decks/{deckId}/topics — danh sách topic trong deck của user.
- POST /api/v1/users/me/decks/{deckId}/topics — tạo topic mới trong deck.
- GET /api/v1/users/me/decks/{deckId}/topics/{topicId} — chi tiết topic.
- PUT /api/v1/users/me/decks/{deckId}/topics/{topicId} — cập nhật topic.
- DELETE /api/v1/users/me/decks/{deckId}/topics/{topicId} — xóa topic kèm toàn bộ card bên trong.

Cards trong deck của user

- GET /api/v1/users/me/decks/{deckId}/cards — danh sách card trong deck; hỗ trợ filter topicId, q, page, limit.
- POST /api/v1/users/me/decks/{deckId}/cards — tạo card mới trong deck. Body đơn giản: topicId, term, translation (bắt buộc); definition, example, pos (tùy chọn). definition lưu vào explanation.vi, example lưu vào examples.en; order tự gán.
- GET /api/v1/users/me/decks/{deckId}/cards/{cardId} — chi tiết card.
- PUT /api/v1/users/me/decks/{deckId}/cards/{cardId} — cập nhật term, translation, definition, example, pos (gửi ít nhất một). Thẻ giữ nguyên topic, không hỗ trợ chuyển nhóm.
- DELETE /api/v1/users/me/decks/{deckId}/cards/{cardId} — xóa card.
- POST /api/v1/users/me/decks/{deckId}/topics/{topicId}/import - Nhập danh sách card từ file Excel vào topic. Yêu cầu đăng nhập bằng Bearer token và phải là chủ sở hữu deck. Định dạng file Excel (header dòng 1): term, translation, explanation_vi, examples_en, pos. Các cột bắt buộc phải có: term và translation.

- GET /api/v1/users/me/decks/{deckId}/topics/{topicId}/export - Xuất danh sách card trong topic ra file Excel. Yêu cầu đăng nhập bằng Bearer token và phải là chủ sở hữu deck.

Tra cứu từ vựng hệ thống

- GET /api/v1/vocabulary/search — tìm card trong các deck hệ thống đã publish theo term (query q bắt buộc, limit tùy chọn 1..50 mặc định 10). Trả về shape phẳng (term, translation, pos, definition, example, sourceCardId) để điền sẵn form tạo/sửa thẻ.

Upload file (S3)

Vòng đời upload 2 bước: (1) xin presigned PUT → (2) client PUT bytes thẳng lên S3. Sau đó gửi url (trả về từ bước 1) trực tiếp vào body của endpoint cập nhật resource; backend validate quyền sở hữu + HeadObject tại đó, không cần bước confirm riêng.

- POST /api/v1/s3/presigned-url — tạo URL PUT ký sẵn (hết hạn 60s) để client upload file trực tiếp lên S3. Body: contentType (bắt buộc), purpose (bắt buộc: shadowing-audio | deck-import | card-image), fileSize (bắt buộc, bytes). Key sinh ở server theo userId; backend không nhận bytes. **fileSize** được bake vào chữ ký → S3 reject upload sai kích thước. Trả về uploadUrl + key + url (public/CDN) + expiresIn. Client dùng url này để gửi kèm khi cập nhật resource. **Phân quyền theo purpose:** **card-image** yêu cầu role **admin** (403 nếu user thường); **shadowing-audio** và **deck-import** cho phép mọi user đã xác thực.

Progress của user

- GET /api/v1/users/me/lessons/{lessonId}/segments-progress — lấy toàn bộ tiến độ segment của một lesson.
- GET /api/v1/users/me/lessons/{lessonId}/segments/{segmentId}/progress — lấy tiến độ một segment.
- PATCH /api/v1/users/me/lessons/{lessonId}/segments/{segmentId}/progress - upsert/cập nhật một phần block dictation hoặc shadowing (nếu chưa có sẽ tự tạo mới tiến độ cho segment này).
- GET /api/v1/users/me/card-states — danh sách trạng thái card; hỗ trợ deckId, topicId, due, starred, hidden, page, limit.
- GET /api/v1/users/me/card-states/{cardId} — lấy trạng thái một card.
- PATCH /api/v1/users/me/card-states/{cardId} — Cập nhật một phần state của một card (srs, flags). Nếu card chưa có state, sẽ tự động tạo mới (bắt buộc phải có deckId và topicId trong body khi tạo mới).

Gamification

Streak

- GET /api/v1/gamification/streak — lấy trạng thái streak của user hiện tại. Trả về { **currentStreak**, **longestStreak**, **lastActiveDayKey**, **activeToday** }. Streak chỉ tăng bởi hành động học thật (hoàn thành segment, review card qua SRS grade, chơi battle) — không có write endpoint.

Profile & Level

- GET /api/v1/gamification/me — hồ sơ gamification của user. Trả về { **totalXp**, **level**, **currentStreak**, **longestStreak**, **xpIntoLevel**, **xpForNextLevel**, **progressPct** }. **progressPct** trong [0, 100]. Nếu chưa có hoạt động, trả về defaults (xp=0, level=1).

Leaderboard

- GET /api/v1/gamification/leaderboard?page=1&limit=20 — bảng xếp hạng global all-time theo tổng XP giảm dần. Trả về { items: [{ rank, userId, name, avatarUrl, totalXp, level }], page, limit, total }. Hòa điểm phân định bằng _id asc. limit tối đa 100. Validation: page ≥ 1, limit trong khoảng từ [1, 100]; lỗi trả 400.

My Rank

- GET /api/v1/gamification/me/rank — thứ hạng của user hiện tại. Trả về { rank, totalXp, totalPlayers }. rank = (số người có totalXp > của mình) + 1. Nếu chưa có hoạt động, rank = tổng số người chơi (xp=0).

Battle (1v1 Real-time)

Giao tiếp chính qua **WebSocket (socket.io)**. REST chỉ dùng để lấy lịch sử sau trận.

Kết nối socket

```
URL: ws://<host> (cùng port với REST server)
Auth: socket.handshake.auth = { token: "<access_token>" }
```

Token phải là **ACCESS** token. Refresh token bị từ chối. Khi token hết hạn cần lấy token mới rồi kết nối lại.

Client -> Server events

Event	Payload	Mô tả
battle:queue:join	{ mode: 'mcq' 'typing' }	Vào hàng chờ ghép trận random
battle:queue:leave	—	Rời hàng chờ
battle:room:create	{ mode: 'mcq' 'typing' }	Tạo phòng invite, nhận room code
battle:room:join	{ code }	Vào phòng bằng mã 6 ký tự
battle:room:leave	—	Host hủy phòng invite đang mở (trước khi có người join)
battle:answer	{ index, answer }	Gửi đáp án (index = số thứ tự câu hiện tại)
battle:rejoin	{ matchId }	Reconnect vào trận đang dở sau khi mất kết nối

Server -> Client events

Event	Payload	Mô tả
battle:room:created	{ code }	Room code 6 ký tự để chia sẻ

Event	Payload	Mô tả
<code>battle:room:left</code>	—	Xác nhận đã hủy phòng invite
<code>battle:question</code>	<code>{ index, total, term, mode, options, deadlineTs }</code>	Câu hỏi mới; <code>deadlineTs</code> là timestamp (ms) khi hết giờ
<code>battle:roundResult</code>	<code>{ index, correctAnswer, scores }</code>	Kết quả sau mỗi câu; <code>scores = { [userId]: number }</code>
<code>battle:finished</code>	<code>{ scores, winnerId, players }</code>	Trận kết thúc; <code>winnerId</code> null = hòa; <code>players = [{userId, score, correctCount}]</code>
<code>battle:opponentDisconnected</code>	<code>{ userId }</code>	Đối thủ mất kết nối, đang trong grace period (15s)
<code>battle:opponentReconnected</code>	<code>{ userId }</code>	Đối thủ reconnect kịp
<code>battle:opponentLeft</code>	<code>{ winnerId }</code>	Đối thủ không quay lại, user hiện tại thắng forfeit
<code>battle:rejoined</code>	<code>{ currentRound, total, term, mode, options, deadlineTs }</code>	Sync state sau khi reconnect thành công
<code>battle:abandoned</code>	—	Cả 2 disconnect → trận bị hủy, không có reward
<code>battle:queue:timeout</code>	—	Không tìm được đối thủ trong 30s
<code>battle:error</code>	<code>{ code }</code>	Lỗi: <code>INVALID_MODE</code> , <code>ROOM_NOT_FOUND</code> , <code>HOST_DISCONNECTED</code> , <code>MATCH_NOT_FOUND</code> , <code>OPPONENT_UNAVAILABLE</code> (đối thủ rời ngay trước khi ghép), <code>MATCH_START_FAILED</code> (không tạo được trận, vd thiếu câu hỏi)

Cơ chế tính điểm

- Đúng: **100 điểm** base + tối đa **50 điểm** speed bonus tỉ lệ với thời gian còn lại
- Sai hoặc hết giờ: **0 điểm**
- Mỗi câu 12 giây. Điểm được tính ngay lúc submit, không phụ thuộc đối thủ.

Reward sau trận

Gate theo 2 lớp — **loại trận** và **nỗ lực**:

- **Streak**: luôn tính cho cả 2 (đã tham gia học thật), bất kể loại trận / kết quả, nếu đã chơi ≥ 1 vòng. Kèm daily bonus +20 XP (1 lần/ngày, idempotent).

- **XP battle (play/win):** chỉ trận `matchType='queue'` (ghép random). Trận `invite` (phòng riêng) **không** có XP battle (chống thông đồng farm) — nhưng vẫn tính streak.
- **Ngưỡng nỗ lực:** chỉ nhận XP battle nếu đúng $\geq$ `minCorrectForReward` (mặc định **3/10** câu). Dưới ngưỡng = streak-only, 0 XP battle (chống vào trả lời bù).
- **Không phạt:** dưới ngưỡng/thua = 0 XP, không trừ.

Hành động	XP	Điều kiện
Streak (+ daily bonus)	+20/ngày	Mọi trận, đã chơi $\geq$ 1 vòng
Tham gia (battle_play)	+15	<code>queue</code> và đúng $\geq$ 3 câu
Thắng (battle_win)	+35	<code>queue</code> và thắng và winner đúng $\geq$ 3 câu

XP idempotent: gọi finalize nhiều lần không bị cộng trùng (`refId=matchId / dayKey`).

REST endpoints

- `GET /api/v1/battle/history` — lịch sử trận đã kết thúc của user (protect). Pagination: `page`, `limit` (tối đa 100). Không trả về questions.
- `GET /api/v1/battle/:id` — chi tiết trận (protect). Chỉ participant xem được (403 nếu không phải). Trả về đầy đủ gồm cả questions.

Admin API

User management

- `GET /api/v1/admin/users` — danh sách user, filter theo status, q, page, limit.
- `GET /api/v1/admin/users/{userId}` — chi tiết một user.
- `PATCH /api/v1/admin/users/{userId}` — cập nhật password.
- `PATCH /api/v1/admin/users/{userId}/status` — khóa / mở khóa user.

Tags management

- `GET /api/v1/admin/tags` — danh sách tag.
- `POST /api/v1/admin/tags` — tạo tag.
- `GET /api/v1/admin/tags/{id}` — chi tiết tag.
- `PUT /api/v1/admin/tags/{id}` — cập nhật tag.
- `DELETE /api/v1/admin/tags/{id}` — xóa tag.

Lessons management

- `GET /api/v1/admin/lessons` — danh sách tất cả lesson, gồm draft, published, archived; hỗ trợ filter status, tagId, ceFrLevelId, mode, q, page, limit.
- `POST /api/v1/admin/lessons` — tạo lesson mới.
- `GET /api/v1/admin/lessons/{lessonId}` — chi tiết lesson bất kể trạng thái.
- `PUT /api/v1/admin/lessons/{lessonId}` — cập nhật lesson.
- `DELETE /api/v1/admin/lessons/{lessonId}` — xóa hoặc archive lesson.

Lesson segments management

- GET /api/v1/admin/lessons/{lessonId}/segments — danh sách segment của lesson.
- POST /api/v1/admin/lessons/{lessonId}/segments — tạo segment mới.
- GET /api/v1/admin/lessons/{lessonId}/segments/{segmentId} — chi tiết segment.
- PUT /api/v1/admin/lessons/{lessonId}/segments/{segmentId} — cập nhật segment như startMs, endMs, transcript, translation.
- DELETE /api/v1/admin/lessons/{lessonId}/segments/{segmentId} — xóa segment.

Decks management

- GET /api/v1/admin/decks — danh sách tất cả deck; hỗ trợ filter tagId, ceFrLevelId, q, page, limit, status.
- POST /api/v1/admin/decks — tạo deck mới.
- GET /api/v1/admin/decks/{deckId} — chi tiết deck.
- PUT /api/v1/admin/decks/{deckId} — cập nhật deck.
- DELETE /api/v1/admin/decks/{deckId} — xóa mềm deck (đổi status sang archived).

Deck topics management

- GET /api/v1/admin/decks/{deckId}/topics — danh sách topic của deck.
- POST /api/v1/admin/decks/{deckId}/topics — tạo topic mới.
- GET /api/v1/admin/decks/{deckId}/topics/{topicId} — chi tiết topic.
- PUT /api/v1/admin/decks/{deckId}/topics/{topicId} — cập nhật topic.
- DELETE /api/v1/admin/decks/{deckId}/topics/{topicId} — xóa topic.
- PATCH /api/v1/admin/decks/{deckId}/topics/reorder — sắp xếp lại topic theo order.

Cards management

- GET /api/v1/admin/decks/{deckId}/cards — danh sách card của deck; hỗ trợ topicId, q, page, limit, pos (loại từ).
- POST /api/v1/admin/decks/{deckId}/cards — tạo card mới.
- GET /api/v1/admin/decks/{deckId}/cards/{cardId} — chi tiết card.
- PUT /api/v1/admin/decks/{deckId}/cards/{cardId} — cập nhật term, pos, phonetics, translation, explanation, examples, imageUrl.
- DELETE /api/v1/admin/decks/{deckId}/cards/{cardId} — xóa card.
- PATCH /api/v1/admin/topics/{topicId}/cards/reorder — sắp xếp lại card theo order trong topic.
- POST /api/v1/admin/decks/{deckId}/topics/{topicId}/import - Import cards vào một topic từ file Excel trên S3.
- GET /api/v1/admin/decks/{deckId}/topics/{topicId}/export - Export cards từ một topic ra file Excel.

Admin API

- POST /api/v1/ai/cards/auto-fill - Dựa vào từ vựng hoặc nghĩa tiếng Việt để tự động sinh ra các trường dữ liệu còn thiếu cho thẻ từ vựng (phát âm, từ loại, giải thích, ví dụ...).